Agent Efficiency Experiment: Results Report

Code Coverage on Spring PetClinic
7 Variants × 3 Runs — What the Data Shows

Tuvium Code Coverage Experiment v2

Tuvium Research

March 2026

Contents

1 What We Ran 2

2 The Stark Contrast 3

3 Two-Axis Results 3

4 Four Key Findings 4

5 The Behavioral Signature: What the Heatmaps Show 5

6 Step-Count Prediction: Does the Math Work? 6

7 What This Experiment Cannot Show 7

8 What Comes Next: Fineract 7

9 The Short Summary 8

1 What We Ran

The task

Write JUnit 5 tests for Spring PetClinic from scratch to maximize code coverage. The agent starts with zero tests. It finishes when it decides the coverage is satisfactory — or when it times out.

Spring PetClinic is the reference Spring Boot application: well-documented, heavily covered in online tutorials, present in training data for all major language models. This matters for interpreting the results (see Section 7).

The setup

- **Model**: Claude Sonnet 4.6
- **Task**: write tests for spring-petclinic (zero initial coverage)
- **Coverage target**: instruction coverage via JaCoCo
- **Build**: `./mvnw clean test jacoco:report`
- **Runs**: 20 sessions total (N=3 per variant, except `simple`: N=2)
- **Tracking**: every tool call logged in order via `agent-journal`

The seven variants

Each variant gives the agent a different level of structured support. They build additively from a minimal prompt up to a two-phase exploration+action structure.

Variant	What the agent has
`simple`	Minimal prompt. No stopping condition. No knowledge. The bare minimum needed to run the task.
`hardened`	Structured prompt with explicit stopping condition and output format. Still no domain knowledge.
`hardened+kb`	Hardened prompt plus a flat knowledge base: Spring test annotation imports, JaCoCo configuration, petclinic package structure.
`hardened+sae`	Hardened prompt plus Structured Analysis and Exploration (SAE): a mandatory pre-analysis pass before writing any tests.
`hardened+skills`	Hardened prompt plus SkillsJars: structured, modular knowledge packages delivered as callable skills.
`hardened+skills+sae`	Skills plus SAE together: structured knowledge delivery and mandatory pre-analysis.
`hardened+skills+sae+forge`	Two-phase design: a deep exploration phase followed by a sustained act phase. The most structured variant.

Table 1: Seven variants tested, ordered from least to most structured.

What we measured

Every tool call was classified into one of 8 semantic states (EXPLORE, READ_KB, READ_SKILL, JAR_INSPECT, WRITE, BUILD, VERIFY, FIX). See the companion document *Reading Agent Behavior* for the full taxonomy.

At the end of each run, a separate judge (T3 TestQualityJudge) evaluated the produced test suite on 6 criteria: test slice selection, assertion quality, error coverage, domain patterns, coverage target selection, and version-aware patterns. Score range: 0.0–1.0. Runs were not scored by the same process that produced the tests (no circular grading).

2 The Stark Contrast

Before the summary tables: two real runs, same model, same task.

	Total calls	BUILD calls	Cost	Duration
hardened+skills+sae (run 17)	69	11	$3.49	26 min
hardened+skills+sae+forge (run 14)	97	**23**	**$5.67**	43 min

Run 17 built once, verified, done. Run 14 built 23 times — six consecutive FIX→BUILD cycles in the middle of the run. The 63% cost difference comes entirely from that spiral.

Both runs produced passing tests. Both scored above 0.85 on the T3 judge. The spiral did not improve the outcome.

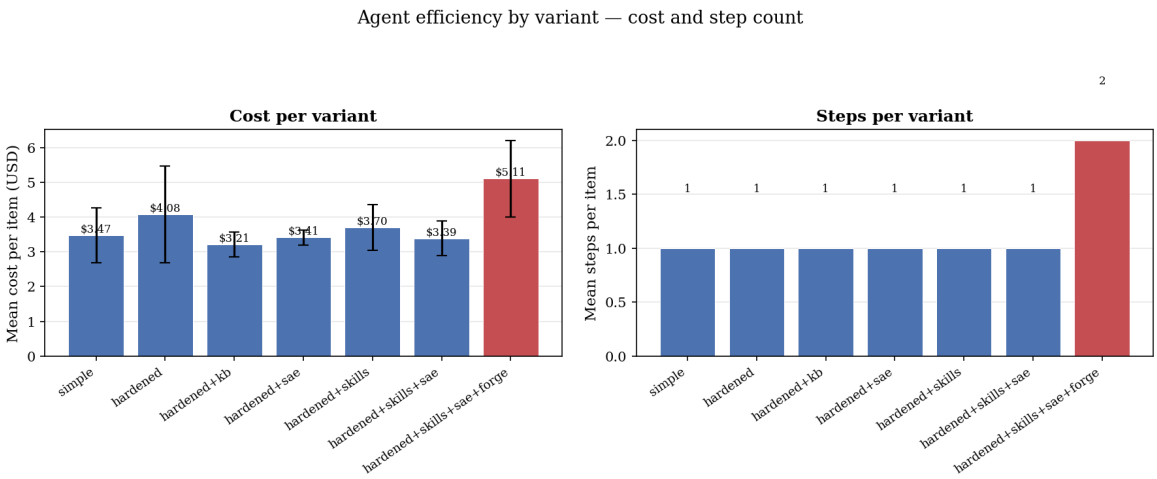


Figure 1: Mean cost (left) and mean step count (right) per variant across all runs. The forge variant (red bar) has the highest mean cost and the highest variance — it can be the cleanest or the most expensive run in the dataset depending on whether it hits a spiral. Error bars = standard deviation across N=3 runs.

3 Two-Axis Results

Efficiency and quality are independent axes. A variant that saves 30% of steps but degrades quality isn't actually saving you money on the goal of *producing good tests*.

Variant	N	Mean steps	±	Mean cost	T3 quality	±
`hardened+skills+sae`	3	75.0	8.7	$3.39	0.850	0.017
`hardened+sae`	3	80.3	12.1	$3.41	0.789	0.035
`hardened+kb`	3	83.3	7.4	$3.21	0.847	0.027
`hardened+skills+sae+forge`	3	95.0	11.1	$5.11	0.878	0.092
`hardened+skills`	3	101.7	10.1	$3.70	0.856	0.025
`hardened`	3	103.0	17.8	$4.08	0.850	0.017
`simple`	2	109.5	13.4	$3.47	0.783	0.071

Table 2: Two-axis comparison: efficiency (steps, cost) and quality (T3 score). Steps = total tool calls per run. T3 quality = mean across N runs ± std. Sorted by mean steps ascending (most efficient first).

The forge paradox: `hardened+skills+sae+forge` ranks 4th on efficiency (95 mean steps) but has the highest mean quality (0.878) *and* the highest standard deviation (0.092). Its best run (run 15: $5.80, T3=0.967) and worst run (run 14: $5.67, 23 builds, spiral) are the extremes of the entire dataset. High ceiling, high floor risk.

4 Four Key Findings

Finding 1: Prompt hardening is the biggest quality driver.

`simple` → `hardened`: +0.067 quality points (0.783 → 0.850), −6% steps. This is the largest single quality jump in the table.

All other knowledge and structure variants build on top of a hardened prompt. The hardened prompt adds a stopping condition and structured output requirements — no external knowledge, just discipline. *Structured prompting beats no structure, regardless of what knowledge you inject.*

Finding 2: SAE drives efficiency at a quality cost.

`hardened` → `hardened+sae`: −22% steps, but quality drops to 0.789 — back near `simple`'s 0.783.

SAE's mandatory pre-analysis pass front-loads exploration and produces a tighter action plan. The agent is more efficient: it writes fewer exploratory tests, makes fewer speculative BUILD attempts. But the plan can be too tight — it scopes out edge cases and domain-specific behaviors that a less constrained agent would discover during exploration. *Efficiency and depth are in tension. SAE resolves it toward efficiency.*

Finding 3: Skills compensate for SAE's quality regression.

`hardened+skills+sae`: −31% steps vs simple, quality = 0.850 — matching `hardened` alone, despite being 28 steps shorter.

Skills provide structured domain knowledge: correct test patterns, annotation imports, petclinic-specific coverage strategies. With skills, the SAE plan is better-informed — it scopes tasks correctly because it has the right vocabulary to scope with. The skill

package functions as a quality stabilizer when combined with SAE. *This is the strongest variant by the efficiency+quality tradeoff.*

Finding 4: KB is a pure efficiency play on known codebases.

`hardened+kb`: -24% steps vs `hardened`, quality $= 0.847$ (flat, $\Delta = -0.003$, within noise).

The knowledge base reduces JAR inspection ($10.1\% \rightarrow 3\%$ of steps roughly) by answering the import questions the agent would otherwise search for. But petclinic is well-known — the model already knows its domain, so KB cannot improve the tests, only reduce friction. *On a novel codebase, KB's quality impact should be larger. This is the central hypothesis for the Fineract follow-up.*

5 The Behavioral Signature: What the Heatmaps Show

The two-axis table shows what happened. The heatmaps show *why*.

The JAR cluster: knowledge friction

When the agent doesn't know which Maven jar provides a Spring test annotation, it inspects jars in `~/.m2/repository` until it finds the class. This is addressable: a knowledge base entry answering the import question eliminates it entirely.

Variant	JAR_INSPECT %	Intervention
`simple`	7.9%	none
`hardened`	11.0%	prompt only (more thorough, no direction)
`hardened+kb`	10.1%	flat KB helps but doesn't eliminate
`hardened+skills`	1.0%	skills answer the import directly
`hardened+skills+sae`	7.1%	SAE reduces JAR by rerouting (different mechanism)

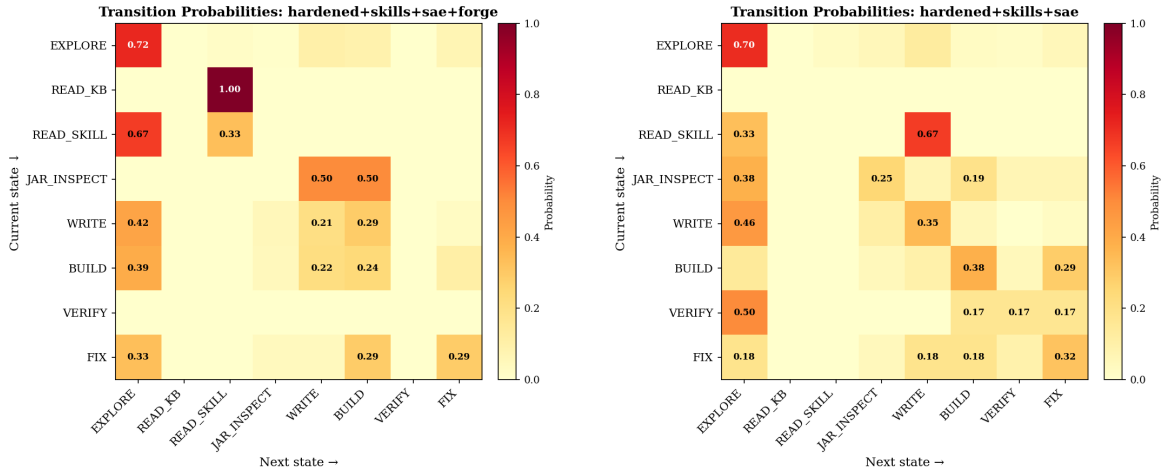
The hardened paradox: adding a hardened prompt *increases* JAR inspection from 7.9% to 11.0%. The hardened prompt makes the agent more thorough about finding correct imports — without providing the answer. Thoroughness without direction = maximum searching.

The FIX/BUILD loop: rework

When tests fail to compile or run, the agent edits (FIX) and rebuilds (BUILD). One or two FIX→BUILD cycles is normal. Six consecutive cycles is a spiral.

The heatmap for `hardened+skills+sae+forge` shows mutual high probability between the BUILD and FIX rows: $P(\text{BUILD} \rightarrow \text{FIX})$ and $P(\text{FIX} \rightarrow \text{BUILD})$ both elevated. Compare to `hardened+skills+sae`: the FIX row is dimmer, fewer cycles, cleaner exit.

The forge variant's two-phase design (deep explore, then sustained act) produces more ambitious initial test writing. When that writing hits a compilation problem, the agent retries aggressively. The recovery is expensive precisely because the initial attempt was so thorough.



Spiral variant (forge): BUILD→FIX and FIX→BUILD cells are orange/red. The agent cycles. Worst run: 23 builds, $5.67.

Clean variant: FIX→BUILD cell is dimmer. Fewer cycles. Best run: 11 builds, $3.49.

Figure 2: The rework loop in the heatmap. Bright mutual probability between BUILD and FIX rows means the agent is cycling. The forge variant's two-phase structure produces this pattern when initial test writing fails.

6 Step-Count Prediction: Does the Math Work?

The Markov framework is only useful if it predicts accurately. We validated using leave-one-out cross-validation: fit the transition matrix P on $N-1$ runs per variant, predict the held-out run's step count from the fundamental matrix $N = (I-Q)^{-1}$.

Cross-validation result: zero mean bias, MAE of 8–20 steps.

Variant	Actual/run	CV Predicted	MAE
`hardened`	103.0	103.0	19.7
`hardened+kb`	83.3	83.3	8.3
`hardened+sae`	80.3	80.3	13.3
`hardened+skills`	101.7	101.7	11.0
`hardened+skills+sae`	75.0	75.0	10.3

The predicted mean matches actual mean for all five variants with N=3 data. The MAE of 8–20 steps represents run-to-run variance, not model error — the model captures the average, not the individual spiral.

Why the prediction is only approximately Markov: A formal second-order test found mean KL divergence of 0.49 bits across 27 (previous, current) state pairs — first-order Markov is formally rejected ($p < 0.001$). The violation is concentrated in recovery transitions: FIX→EXPLORE has 2.10 bits of second-order information. Knowing whether an agent arrived at FIX from its first build attempt or its sixth dramatically changes what it explores next.

But over a 100-step trajectory, recovery transitions happen a handful of times and the errors cancel. Hence: zero mean bias in cross-validation despite technically non-Markov data.

7 What This Experiment Cannot Show

The petclinic ceiling.

Spring PetClinic is one of the most documented Spring Boot applications in existence. All variants reach approximately 92–94% instruction coverage. Coverage is not a discriminator here — the model knows this domain.

Consequences:

- Knowledge injection (KB, skills) improves *efficiency* (fewer steps) but cannot improve *outcome quality* beyond the model's existing knowledge of petclinic.
- The T3 quality scores are meaningful — they reflect how well-structured the tests are — but they cannot reflect domain-knowledge improvement on a domain the model already knows.
- To prove that knowledge injection improves *outcomes* on novel domains, we need a domain the model does not know well.

What petclinic does prove:

- Prompt hardening alone improves quality and reduces steps. This holds even when the model knows the domain perfectly.
- Structured execution (SAE) reliably reduces steps on any codebase — the pre-analysis benefit is architectural, not domain-specific.
- The JAR cluster is addressable via knowledge injection. Skills reduce it from 11% to 1% regardless of domain familiarity.
- The FIX/BUILD spiral risk is variant-specific. Forge has it; `hardened+skills+sae` does not.

What petclinic is: the negative control. It establishes the lower bound on knowledge injection's quality effect (zero improvement on known domains) and the baseline behavioral signatures.

8 What Comes Next: Fineract

The next experiment targets **Apache Fineract** (banking platform, Spring Boot 3.2.2, Gradle 8.x). The `fineract-loan` module covers a bounded loan lifecycle domain — disbursement, repayment schedules, interest calculations, charge types.

This domain is not well-represented in model training data. The model has seen petclinic; it has not seen Fineract's loan allocation rules.

What Fineract tests

The central hypothesis: **on a novel domain, knowledge injection improves test quality, not just test efficiency.**

The KB will contain 4 domain files:

- `loan-lifecycle.md` — disbursement → repayment → closure states
- `test-patterns.md` — Spring Boot 3 test patterns, JPA, H2 in-memory

- `loan-charge-types.md` — fee, penalty, and upfront charge classification
- `assertion-guide.md` — how to write assertions for financial calculations

Fineract already has tests at roughly 40–65% line coverage (module-dependent). The task is to improve from that baseline, not write from scratch. This is harder: the agent must understand what tests already exist and write tests that extend coverage without duplicating them.

The experimental matrix

6 variants × 3 runs = 18 sessions:

1. `simple` — baseline
2. `hardened` — structured prompt
3. `hardened+kb` — flat KB with loan domain files
4. `hardened+skills` — SkillsJars with same content
5. `hardened+skills+sae` — skills + pre-analysis
6. `hardened+skills+sae+forge` — two-phase

Success criterion: at least one KB variant (+kb or +skills) achieves ≥10% T2-improvement (coverage gain above simple baseline). If yes: knowledge injection proves outcome improvement on novel domains. If no: efficiency is the only measurable benefit, and the thesis requires refinement.

9 The Short Summary

What the v2 experiment shows:

1. **Prompt hardening is the single biggest quality driver.** +0.067 quality, −6% steps. Free gain from structural discipline.
2. **SAE is efficient but shallow.** −22% steps, quality back near baseline. Tight plans miss edge cases.
3. **Skills fix SAE's quality regression.** `hardened+skills+sae`: −31% steps, quality matches `hardened`. Best tradeoff in the experiment.
4. **KB and skills reduce JAR friction but not quality on known codebases.** On petclinic: efficiency gain, quality flat. On novel domains: untested.
5. **Forge is high variance.** Best quality ceiling, worst spiral risk. Its two-phase design amplifies both success and failure.
6. **The Markov model predicts step counts accurately.** Zero mean bias in cross-validation despite formal rejection of first-order. Recovery transitions are non-Markov; averages are robust.

What comes next: Apache Fineract — a novel banking domain — to test whether the quality claim holds beyond a domain the model already knows.